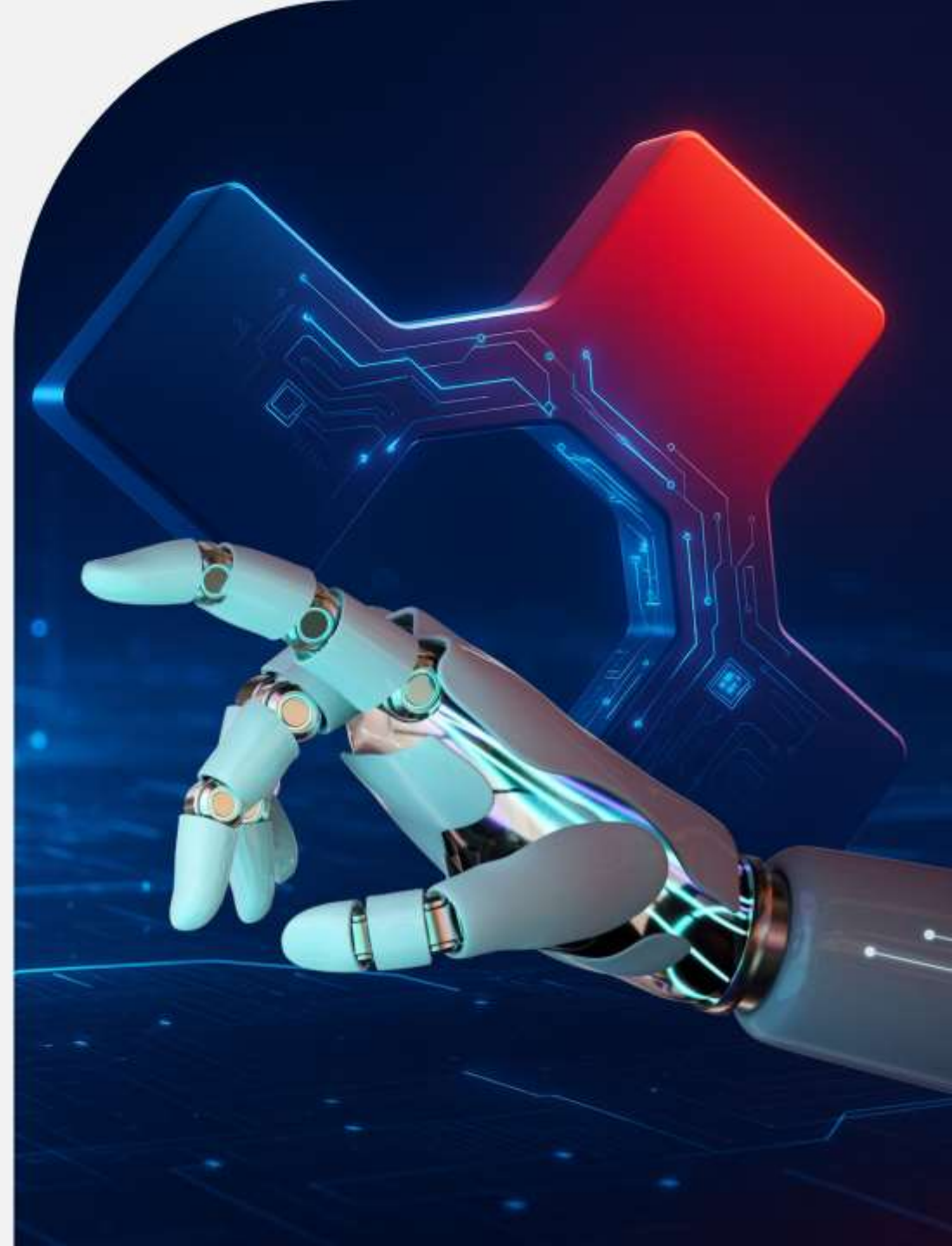
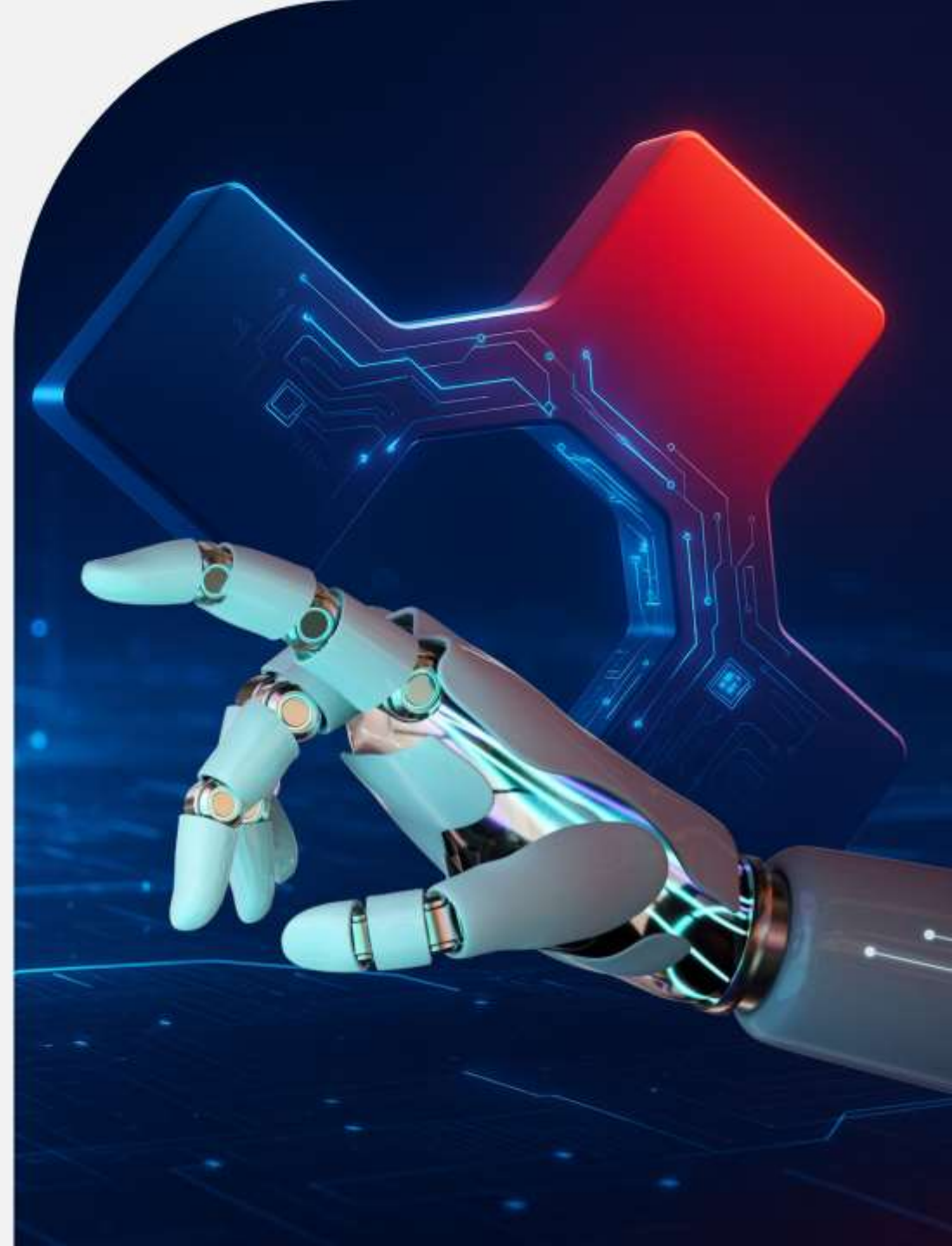


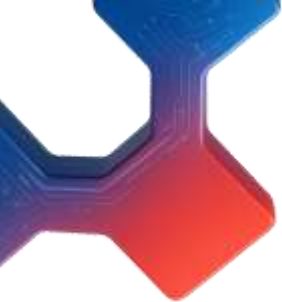


Núcleo de Capacitação em Inteligência Artificial



Aula 15





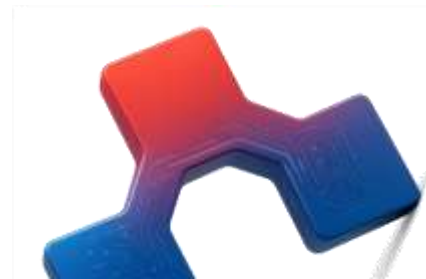
1. Introdução

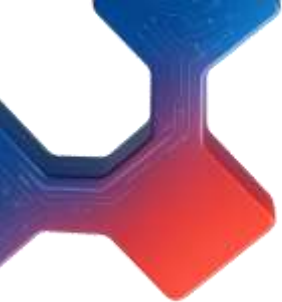
1. Trabalhamos no problema de previsão médio de casas, e usamos os seguintes modelos:

- **LinearRegression**
- **DecisionTreeRegressor**
- **RandomForestRegressor**
- **Support Vector Regression.**

2. Trabalhamos no problema classificação de imagens de dígitos , e usamos os seguintes modelos:

- **SGDClassifier**
- **KNeighborsClassifier**
- **RandomForestRegressor**
- **Support Vector Classification .**





1. Introdução

Abordaremos duas maneiras diferentes de treinar uma **regressão linear**:

- Usando uma equação direta de forma fechada que calcula os parâmetros do modelo que melhor se ajustam ao modelo no conjunto de treinamento (ou seja, os parâmetros do modelo que minimizam a função de custo no conjunto de treinamento).
- Usando uma abordagem de otimização, chamada **gradiente descendente**, os parâmetros do modelo são ajustados gradualmente para minimizar a função de custo no conjunto de treinamento, eventualmente convergindo para o mesmo conjunto de parâmetros obtido pelo método de solução analítica.

1. Gradiente descendente em batch (em lote)

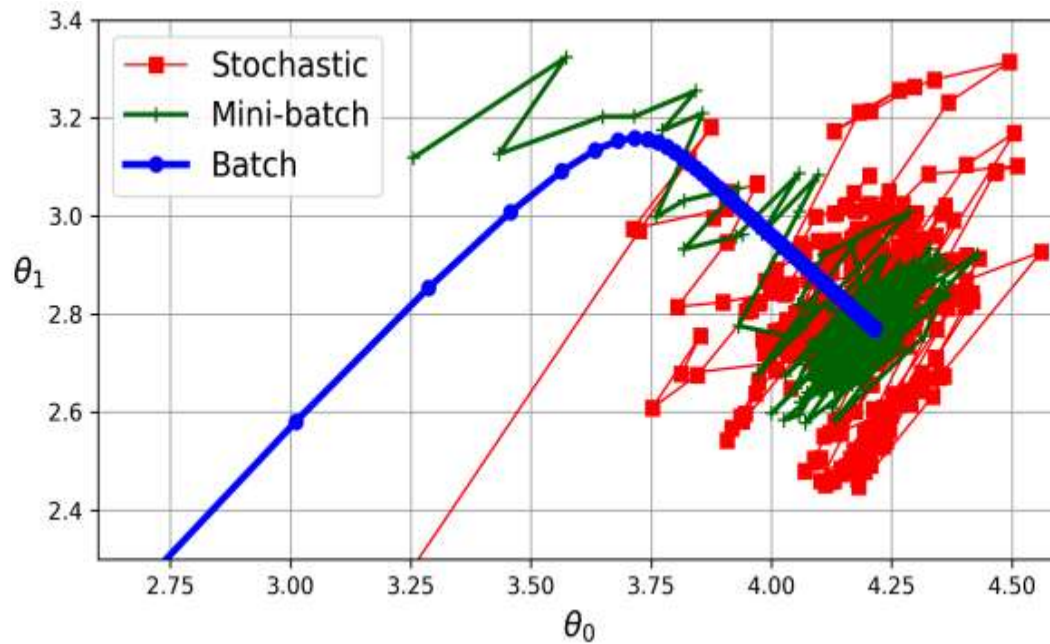
2. Gradiente descendente estocástico

3. Gradiente descendente em mini-batch.

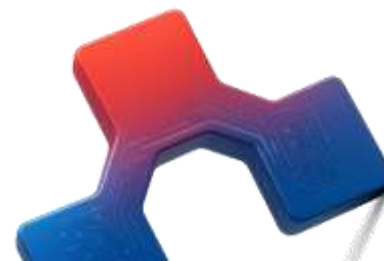


2. Gradiente Descendente em mini-batch

- Em cada etapa o gradiente descendente em mini-batch calcula os gradientes em pequenos conjuntos aleatórios de instância chamados mini-batches.

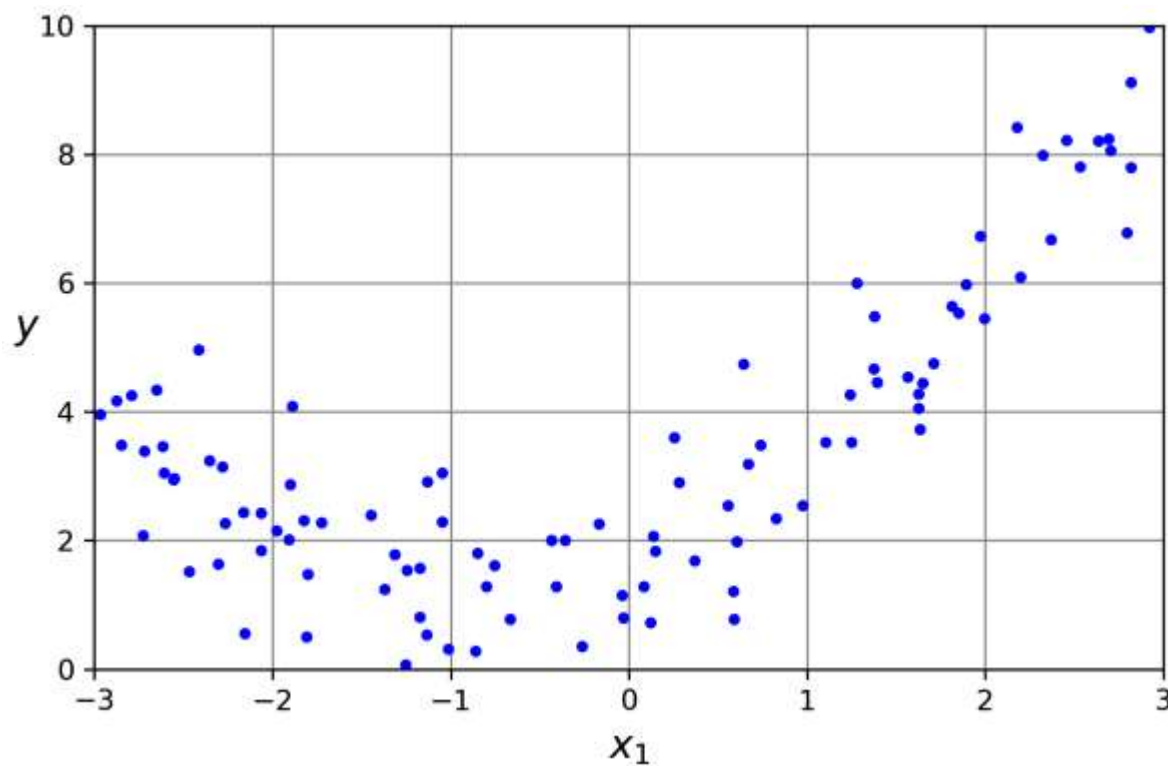


Algoritmo	Grande m	Suporte out-of-core	Grande n	Hiperparâmetros	Escalonamento requerido	Scikit-Learn
Método dos Mínimos Quadrados	Rápido	Não	Lento	0	Não	LinearRegression
GD Lote	Lento	Não	Rápido	2	Sim	n/a
GD Estocástico	Rápido	Sim	Rápido	≥ 2	Sim	SGDRegressor
GD Minilote	Rápido	Sim	Rápido	≥ 2	Sim	SGDRegressor



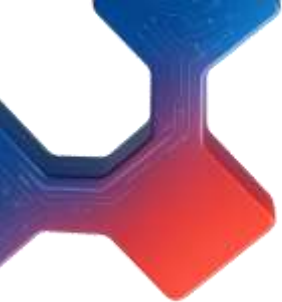
2. Regressão Polinomial

- E se os dados se comportarem de modo não-linear ?



- Curiosamente podemos usar um modelo linear para ajustar dados não lineares.
- Adicionar as potências de cada características novas e treinar um modelo linear nesse conjunto dilatado de características.
- **Regressão polinomial.**



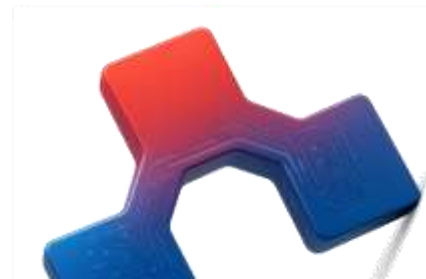


2. Regressão Polinomial

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929,  0.56664654])
```

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

- Usar a classe **PolynomialFeatures** do Scikit-Learn para transformar os dados de treinamento , adicionando (polinômio do segundo grau) de cada característica no conjunto de treinamento com uma nova característica.
- O dados ficaram “alargados” agora podemos ajustar um modelo **LinearRegression** a esses dados de treinamento.



2. Curvas de Aprendizado

```
from sklearn.model_selection import learning_curve

train_sizes, train_scores, valid_scores = learning_curve(
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")

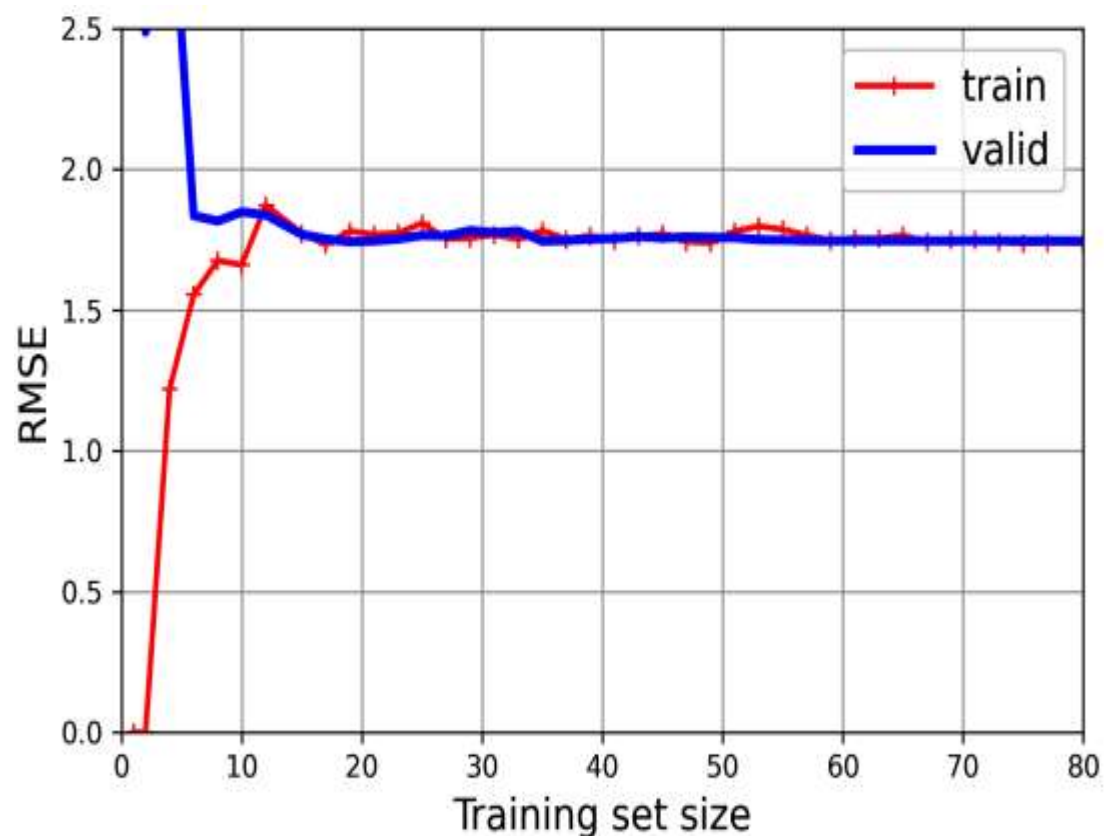
train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)

plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")
[...] # beautify the figure: add labels, axis, grid, and legend
plt.show()
```

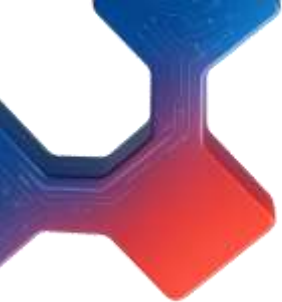
- Curvas de aprendizado são gráficos de desempenho do modelo no conjunto de treinamento e no conjunto de validação como função do tamanho do conjunto de treinamento (ou da iteração do treinamento).
- Para gerar os gráficos, basta treinar o modelo diversas vezes em subconjuntos de tamanhos diferentes no conjunto de treinamento.



2. Curvas de Aprendizado



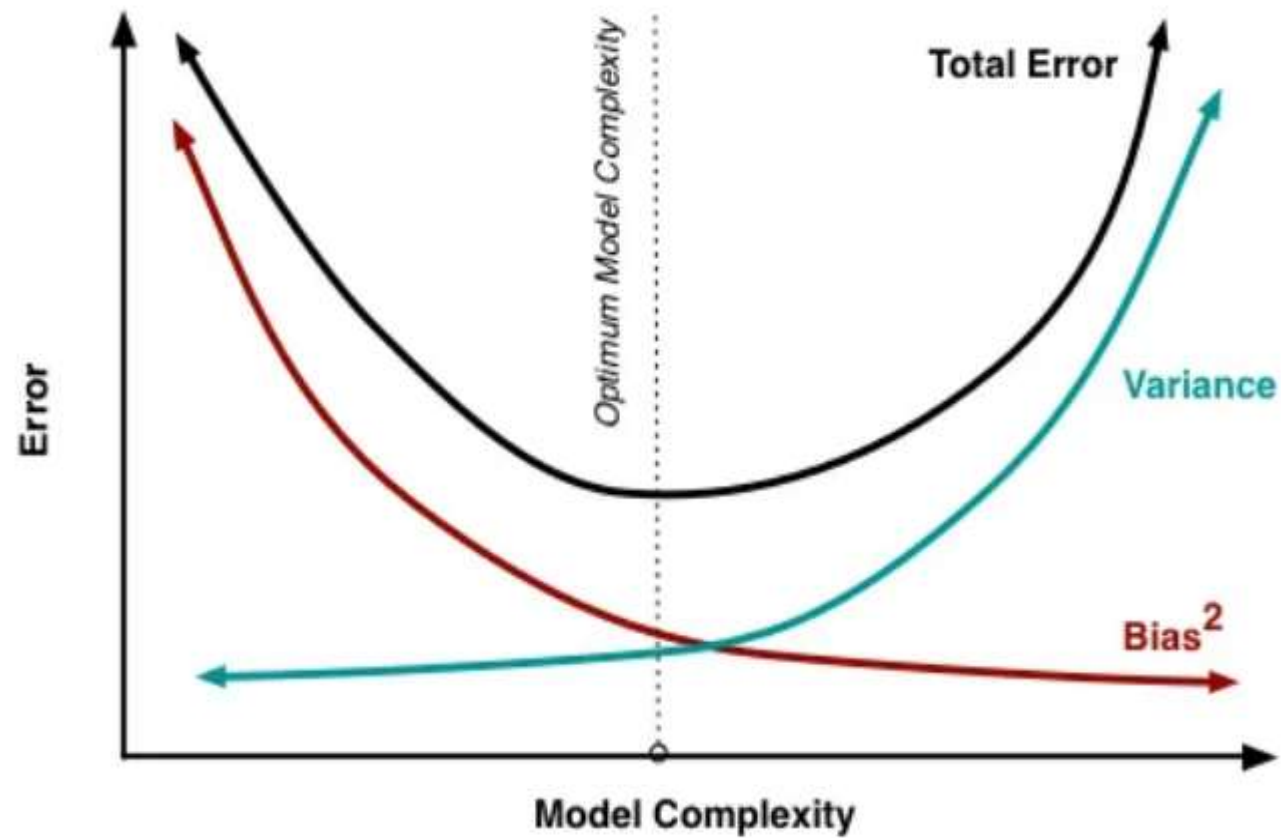
- **Erro de treinamento:** Com poucas instâncias, o modelo se ajusta perfeitamente, mas à medida que mais dados são adicionados, não consegue capturar toda a complexidade dos dados não lineares, e o erro atinge um platô.
- **Erro de validação:** Inicialmente alto devido à incapacidade de generalização com poucos dados, diminui à medida que mais exemplos são apresentados, mas também se estabiliza em um platô por causa da limitação do modelo linear.

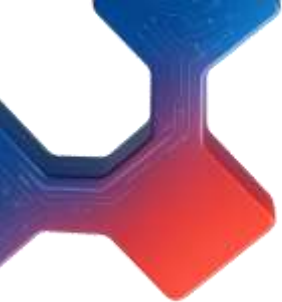


3. O trade-off do Viés/ Variância

- **Viés** : Esta parte do erro de generalização se deve a suposições erradas, como supor que os dados são lineares quando na verdade são quadráticos. É provável que um modelo com um viés alto se subajuste aos dados de treinamento.
- **Variância** : Esta parte se deve à sensibilidade excessiva do modelo a pequenas variações nos dados de treinamento. É provável que um modelo com muitos graus de liberdade tenha uma variância alta, logo ele se sobreajusta aos dados de treinamento.
- **Erro Irredutível** : Esta parte é devido ao ruído dos próprios dados. A única maneira de reduzir esta parte do erro é limpar os dados (por exemplo, corrigir as fontes de dados, como sensores quebrados, ou detectar e remover outliers).

3. O trade-off do Viés/ Variância





4. Modelos Lineares Regularizados

- Uma boa forma de reduzir o sobreajuste é regularizar o modelo (isso é, restringi-lo): quanto menor for o grau de liberdade, mais difícil será para sobreajustar os dados.
- Qual é uma forma simples de regularizar um modelo polinomial ?
- Tipos de regularização:
 - 1. Regressão de Ridge**
 - 2. Regressão de Lasso**
 - 3. Regressão Elastic net**

Ou seja, vamos estudar três maneiras de restringir os pesos.



4.1. Regressão de Ridge

- A regressão de Ridge é uma versão regularizada da regressão linear: um termo de regularização é adicionado à função de custo.

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \underbrace{\frac{\alpha}{m} \sum_{i=1}^n \theta_i^2}_{\text{Termo de regularização}}$$

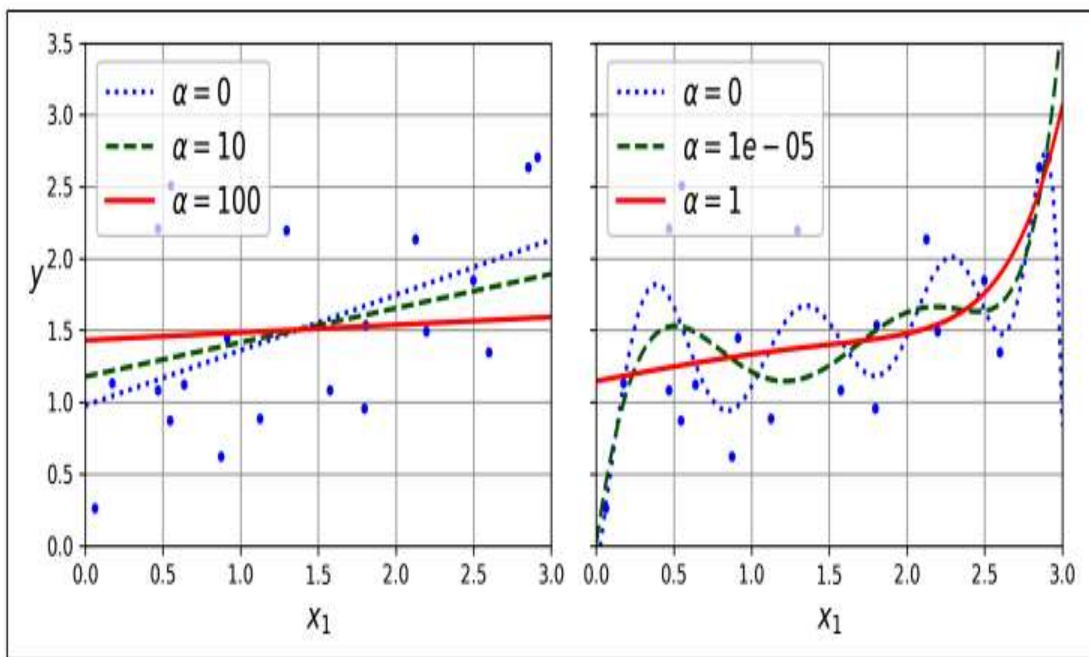
hiperparâmetro que controla o quanto você deseja regularizar o modelo

- Quando $\alpha=0$ a regressão de Ridge é somente a regressão linear.
- Quando o α for muito grande, todos os pesos acabarão muito próximo de zero e o resultado será uma linha achatada que passa pela média dos dados.

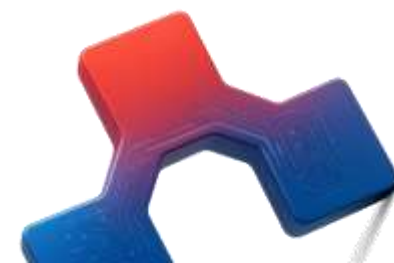


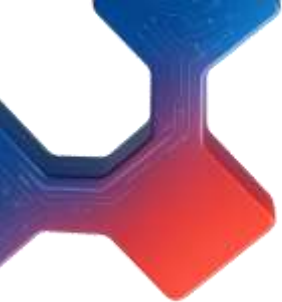
4.1. Regressão de Ridge

- É fundamental escalonar os dados (por exemplo, usando a **StandardScaler**) antes de executar a regressão de Ridge, pois ela é sensível ao escalonamento das características de entrada. **Isso vale para a maioria dos modelos regularizados.**



- A figura mostra diversos modelos de Ridge treinados em alguns dados lineares usando diferentes valores α . À esquerda, há modelos de Ridge simples, que resultam em previsões lineares. À direita, os dados são expandidos primeiro usando a **PolynomialFeatures (degree=10)**, depois são escalonados usando a **StandardScaler** e, finalmente os modelos de Ridge são aplicados às características restantes: Trata-se de uma regressão polinomial com regularização de Ridge.





4.1 Regressão de Ridge

- Assim como na regressão linear, podemos fazer a regressão de Ridge por meio de uma equação de forma fechada ou pelo gradiente descendente. Os prós e contras são os mesmos.

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X} + \alpha \mathbf{A})^{-1} \mathbf{X}^T \mathbf{y}$$

- A seguir vamos mostrar como executar a regressão de Ridge com Scikit-Learn usando uma solução de forma fechada.

4.1. Regressão de Ridge

```
>>> from sklearn.linear_model import Ridge
>>> ridge_reg = Ridge(alpha=0.1, solver="cholesky")
>>> ridge_reg.fit(X, y)
>>> ridge_reg.predict([[1.5]])
array([[1.55325833]])
```

```
>>> sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
...                          max_iter=1000, eta0=0.01, random_state=42)
...
>>> sgd_reg.fit(X, y.ravel()) # y.ravel() because fit() expects 1D targets
>>> sgd_reg.predict([[1.5]])
array([1.55302613])
```

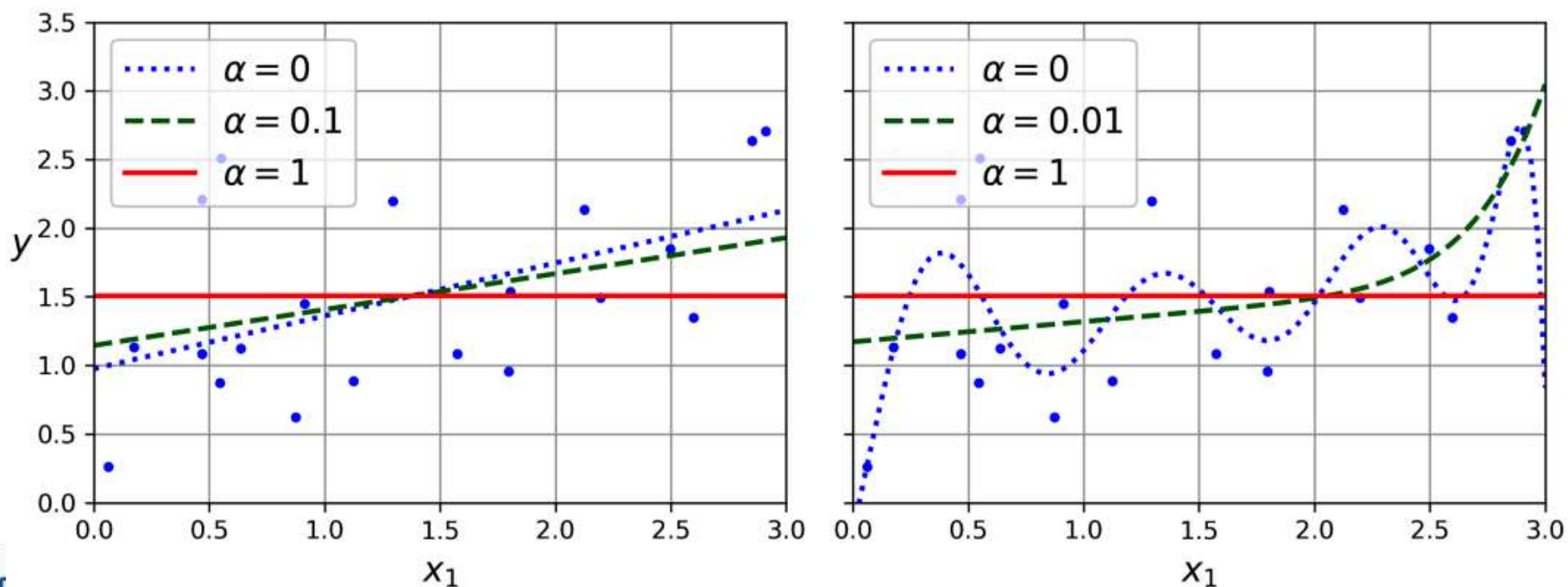
- O hiperparâmetro **penalty** define o tipo de termo de regularização a ser usado. A especificação de "12" indica que você quer que o SGD acrescente um termo de regularização à função de custo igual a metade do quadrado da ℓ_2 do vetor de peso: isso é justamente a regressão de Ridge.



4.2. Regressão de Lasso

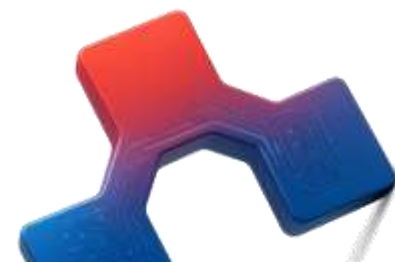
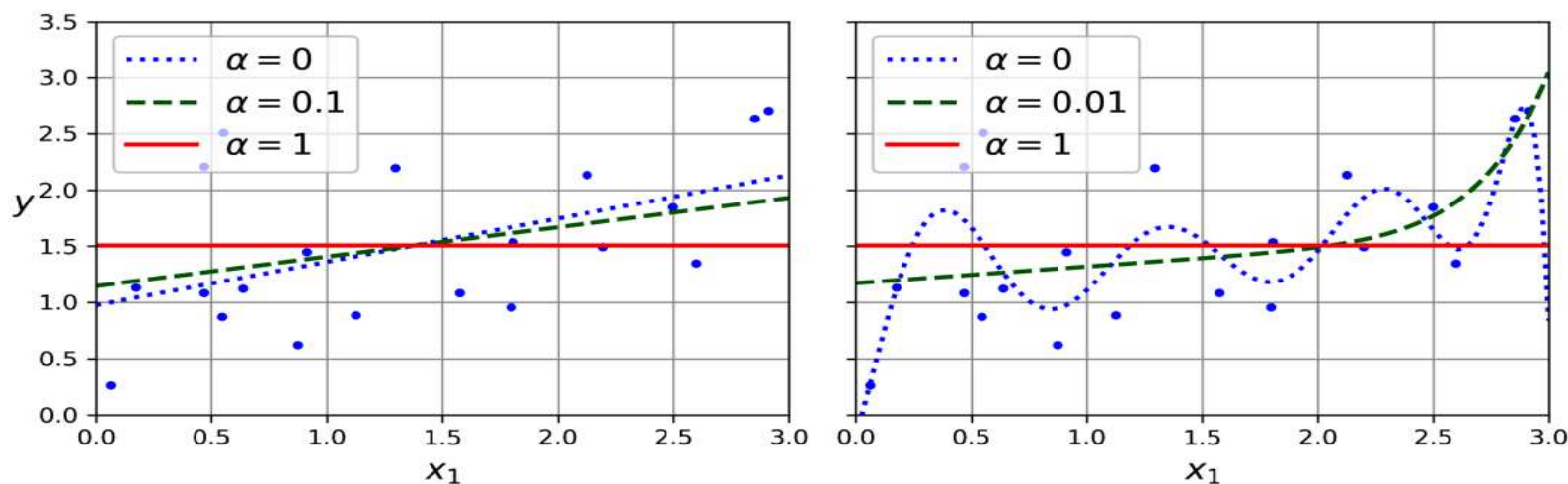
- O método de regressão Lasso é uma outra versão regularizada da regressão linear: assim com a regressão ridge, ela adiciona um termo de regularização à função de custo, mas usa a norma ℓ_1 do vetor de peso, em vez de metade do primeiro quadrado da norma ℓ_2 .

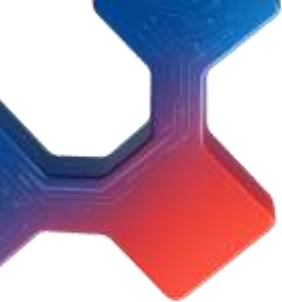
$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + 2\alpha \sum_{i=1}^n |\theta_i|$$



4.2. Regressão de Lasso

- Uma particularidade importante da regressão de lasso é que ela tende a suprimir os pesos das características menos importantes (ou seja, defini-los como zero). Por exemplo, a linha tracejada no gráfico à direita da figura abaixo (com α igual a 10^{-7}) parece quadrática, quase linear: todos os pesos para as características polinomiais do alto grau são iguais a zero. Grosso modo, a regressão de lasso executa automaticamente a seleção de características e gera como saída um modelo esparso.

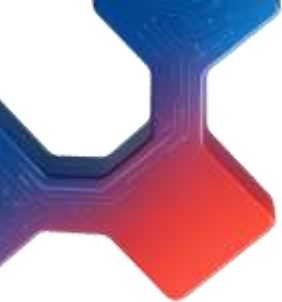




4.2. Regressão de Lasso

1. O código usa a **regressão Lasso**, que é uma regressão linear com regularização ℓ_1 , responsável por gerar **modelos esparsos**.
2. Primeiro, importa-se a classe Lasso.
3. Em seguida, cria-se o modelo definindo $\alpha = 0.1$, que controla a força da regularização. Quanto maior α , mais pesos são forçados a zero.
4. O modelo é treinado com os dados X e y usando o método `.fit()`.
5. Por fim, `.predict([[1.5]])` retorna a previsão para uma nova entrada 1.5, que neste caso é aproximadamente 1.5379.

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.53788174])
```

4.3. Elastic net

- A Elastic net é um meio-termo entre a regressão de Rigde e a regressão de lasso. O termo de regularização é uma simples combinação dos termos de regularização de Rigde e de Lasso, e podemos controlar a relação dessa combinação em r . Quando $r=0$, a elastic net é equivalente à regressão de Rigde, e quando $r=1$ é equivalente a regressão de lasso.

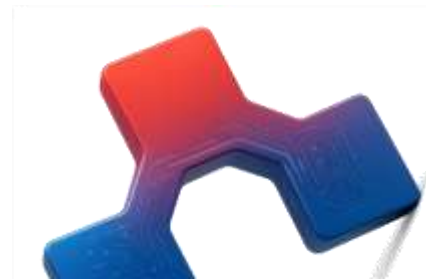
$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r(2\alpha \sum_{i=1}^n |\theta_i|) + (1 - r)\left(\frac{\alpha}{m} \sum_{i=1}^n \theta_i^2\right)$$

- Mas, quando utilizar uma simples regressão linear (por exemplo, sem qualquer regularização), de Rigde, de lasso ou uma regressão elastic net ? Quase sempre é preferível ter pelo menos um pouco de regularização, então você deve evitar uma regressão linear simples. A regressão de Rigde é um bom padrão, mas, se suspeitar de que apenas algumas características são úteis, deve optar pela regressão de lasso ou pela elastic net, pois elas tendem a zero os pesos das características inúteis.

4.3. Elastic net

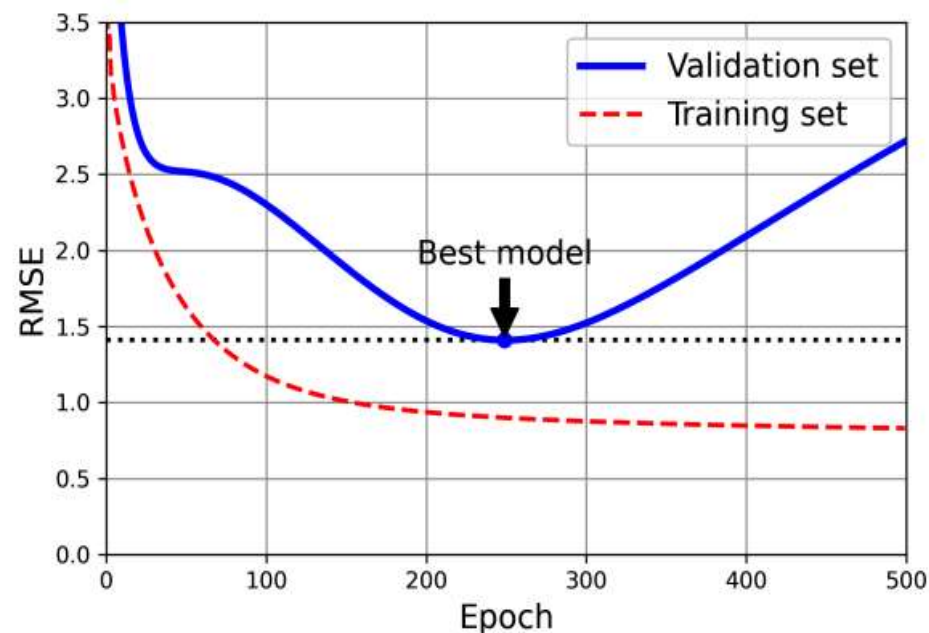
- O código usa o **Elastic Net**, um modelo de regressão linear com regularização que combina Lasso (ℓ_1) e Ridge (ℓ_2). Primeiro importa-se a classe, depois cria-se o modelo definindo a força da regularização ($\alpha = \mathbf{0.1}$) e a proporção entre ℓ_1 e ℓ_2 (l1_ratio = 0.5). Em seguida, o modelo é treinado com os dados X e y . Por fim, ao prever para a entrada 1.5, o resultado é aproximadamente 1.54

```
>>> from sklearn.linear_model import ElasticNet
>>> elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
>>> elastic_net.fit(X, y)
>>> elastic_net.predict([[1.5]])
array([1.54333232])
```



4.3. Early stopping

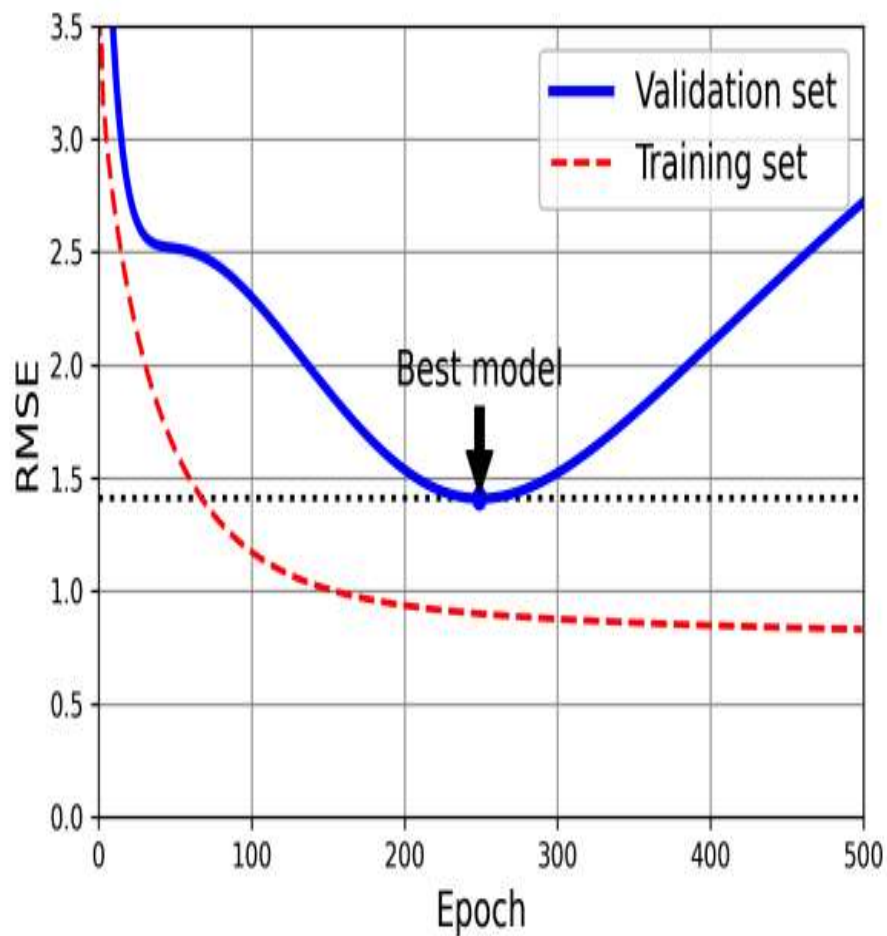
- Um jeito bem diferente de regularizar algoritmos de aprendizado iterativo, como o gradiente descendente, **é parar o treinamento assim que o erro de validação atingir o mínimo. Isso se chama early stopping.**



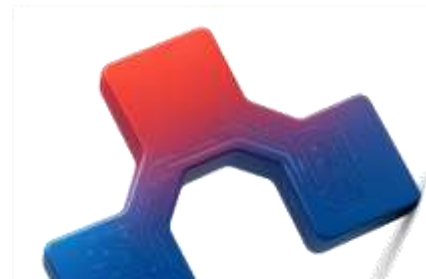
- A figura ao lado mostra um modelo sendo treinado com um gradiente descendente em batch. À medida que as épocas passam pelo algoritmo que aprende, seu erro de predição (RMSE) no conjunto de treinamento diminui, junto com seu erro de predição no conjunto de validação.
- Depois de um tempo, o erro de validação para de diminuir e começa a voltar ao normal. Isso indica que o modelo começou a se sobreajustar aos dados de treinamento.



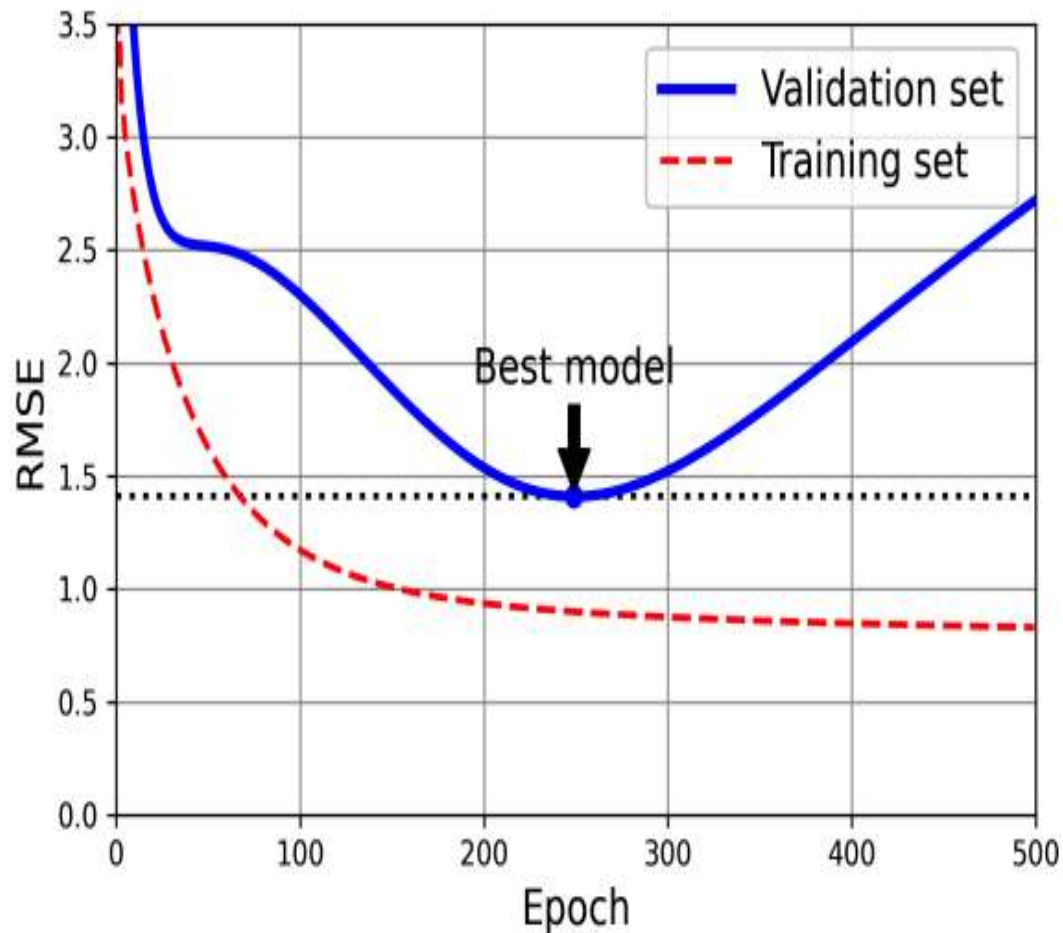
4.3 Early stopping



- Com o **early stopping**, você interrompe o treinamento assim que o erro de validação atingir o mínimo. É uma técnica de regularização tão simples e eficiente que o Geoffrey Hinton a chamou de "um senhor almoço grátis".



4.3 Early stopping

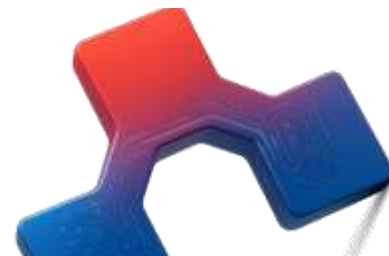


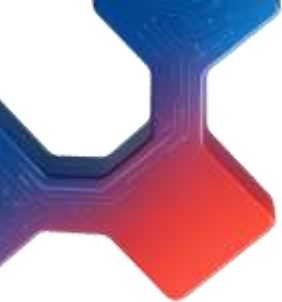
```
from copy import deepcopy
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

X_train, y_train, X_valid, y_valid = [...] # split the quadratic dataset

preprocessing = make_pipeline(PolynomialFeatures(degree=90, include_bias=False),
                              StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
n_epochs = 500
best_valid_rmse = float('inf')

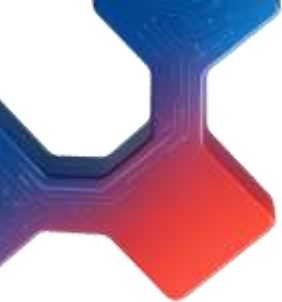
for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = mean_squared_error(y_valid, y_valid_predict, squared=False)
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)
```





5. Regressão logística

- A regressão logística é comumente usada para **estimar a probabilidade** de uma instância pertencer a uma classe específica (por exemplo, qual é a probabilidade de que este e-mail seja um spam?). Caso a probabilidade estimada seja maior que 50% , o modelo prediz que a instância pertence a classe (denominada classe positiva, rotulada como "1"); senão, prevê que não pertence (por exemplo, pertence à classe negativa, rotulada como "0"). Isso torna um classificar binário.
- Assim como um modelo de regressão linear, um modelo de regressão logística calcula a soma ponderada das características de entrada (mais o viés), contudo, em vez de gerar a saída do resultado diretamente como um modelo de regressão linear, ele gera a saída logística desse resultado.



5. Regressão logística

- A regressão logística é comumente usada para **estimar a probabilidade** de uma instância pertencer a uma classe específica (por exemplo, qual é a probabilidade de que este e-mail seja um spam?). Caso a probabilidade estimada seja maior que 50% , o modelo prediz que a instância pertence a classe (denominada classe positiva, rotulada como "1"); senão, prevê que não pertence (por exemplo, pertence à classe negativa, rotulada como "0"). Isso torna um classificar binário.
- A regressão logística é definida por :

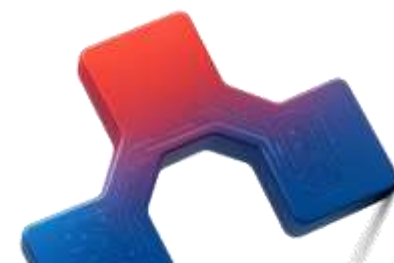
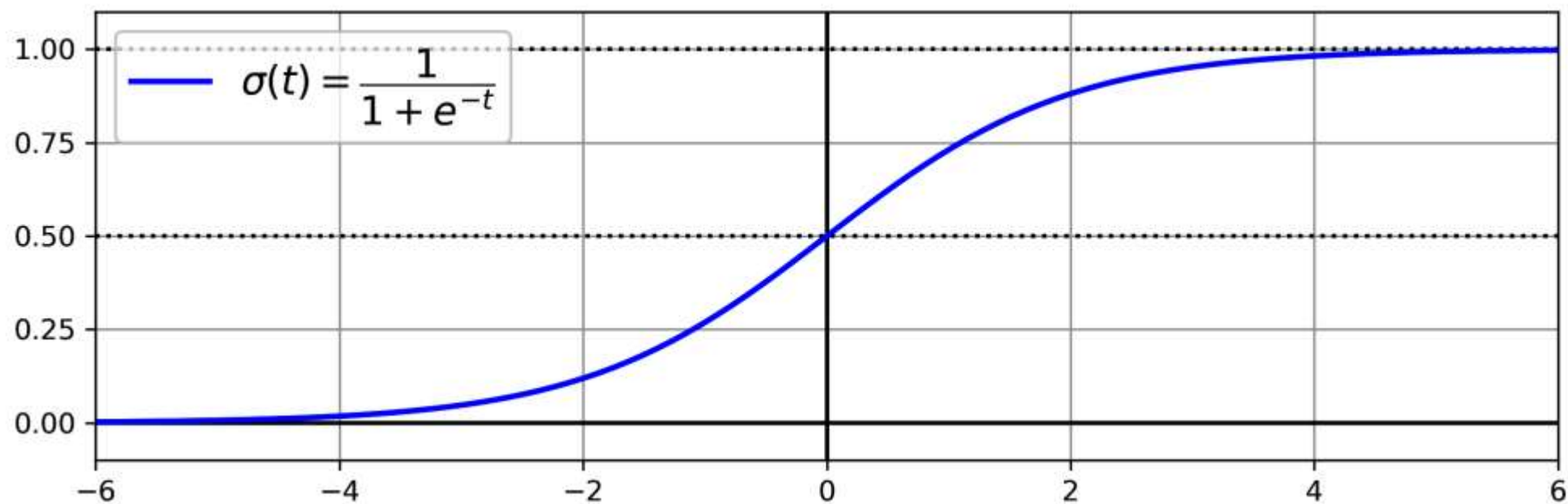
onde

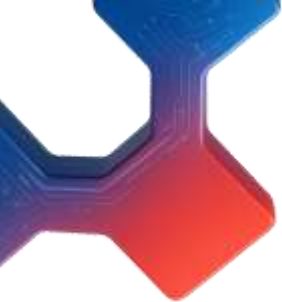
$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\theta^T \mathbf{x})$$

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$



5. Regressão logística



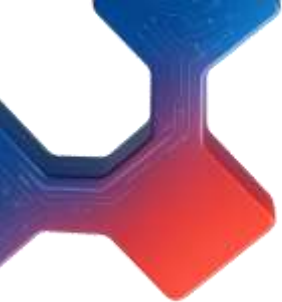


5. Regressão logística

- Uma vez que o modelo de regressão logística estimou a probabilidade $\hat{p}$ de que uma instância pertença à classe positiva, ele consegue prever $\hat{y}$ facilmente da seguinte forma:

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

Observe que $\sigma(t) < 0.5$ quando $t < 0$ e $\sigma(t) \geq 0.5$ quando $t \geq 0$, logo um modelo de regressão logística prevê 1 se $x^T\theta$ for positiva e 0 se for negativa.

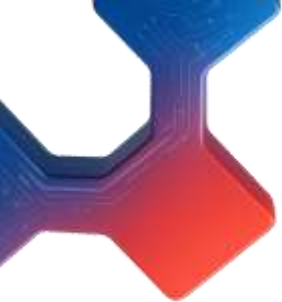


5.1 Treinamento e função de custo

- Como esse modelo é treinado ? O objetivo do treinamento é definir o vetor de parâmetro θ de modo que o modelo estime probabilidades altas para instâncias positivas ($y=1$) e probabilidades baixas para instâncias negativas ($y=0$). A **função de custo** incorpora essa ideia mostrada na equação a seguir para uma única instância de treinamento $\mathbf{x}$.

$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1 - \hat{p}) & \text{if } y = 0 \end{cases}$$

- Você sabe me dizer o motivo de essa função de custo fazer sentido ?

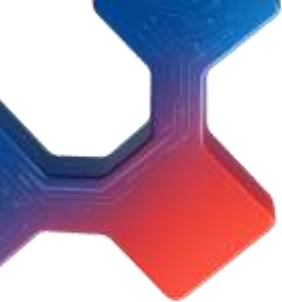


5.1 Treinamento e função de custo

- A função de custo em todo o conjunto de treinamento é o custo médio em todas as instâncias de treinamento. Pode ser escrita em uma única expressão que se chama **log loss** mostra a seguir:

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

- A má notícia é que não existe uma equação de forma fechada conhecida para calcular o valor de $\boldsymbol{\theta}$ que minimiza essa função de custo (não existe equivalente da equação normal). Já a boa notícia é que essa função de custo é convexa; assim sendo, o gradiente descendente (ou qualquer outro algoritmo de otimização) garante o mínimo global (se a taxa de aprendizado não for muito grande e você esperar o suficiente).



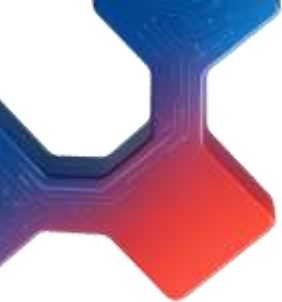
5.1 Treinamento e função de custo

- As derivadas parciais da função custo em relação ao j-ésimo parâmetro do modelo são fornecido pela igualdade a seguir:

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \left(\sigma(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

- Essa equação é bastante semelhante a da regressão linear; ela calcula o erro de predição para cada instância, o multiplica pelo valor da j-ésima característica e depois calcula a média em relação a todas instâncias de treinamento. Uma vez que você tem o vetor de gradiente contendo todas as derivadas parciais, consegue usá-lo no algoritmo do gradiente descendente. É isto: agora já sabemos como treinar um modelo de regressão logística.





5.2. Regressão softmax

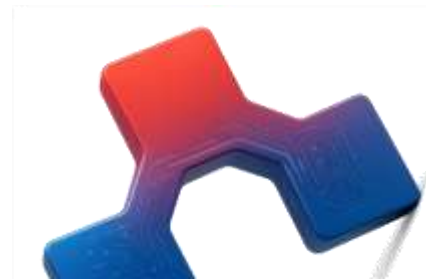
- Dada uma instância $\mathbf{x}$, o modelo de regressão softmax calcula um **score** $s_k(\mathbf{x})$ para cada classe $\mathbf{k}$, depois estima a probabilidade de cada classe aplicando a função softmax aos **scores**.

$$s_k(\mathbf{x}) = \left(\boldsymbol{\theta}^{(k)} \right)^T \mathbf{x}$$

repare que cada classe tem seu próprio vetor de parâmetros dedicado $\boldsymbol{\theta}^k$. Todos esses vetores são normalmente armazenados como linhas em uma matriz de parâmetros.

- A função softmax :

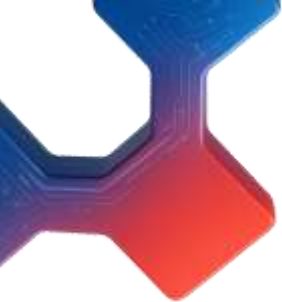
$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$



5.2. Regressão softmax

- Na equação anterior temos que:
 1. K é o número de classe
 2. $s(x)$ é um vetor que contém os scores de cada classe para instância x .
 3. $\sigma(s(x))_k$ é a probabilidade estimada de que a instância à classe k , considerando os scores de cada classe para essa instância.
- Assim como o classificador de regressão logística, o classificador de regressão softmax prevê a classe com maior probabilidade estimada conforme:

$$\hat{y} = \operatorname{argmax}_k \sigma(s(\mathbf{x}))_k = \operatorname{argmax}_k s_k(\mathbf{x}) = \operatorname{argmax}_k \left(\left(\boldsymbol{\theta}^{(k)} \right)^T \mathbf{x} \right)$$

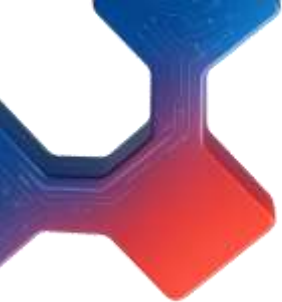


5.2. Regressão softmax

- Olhando para o treinamento. O objetivo é ter um modelo que estima uma probabilidade alta para a classe-alvo (consequentemente uma probabilidade baixa para as outras classes). Minimizar a função de custo mostrada na equação a seguir chamada **entropia cruzada**, pode ajudar a alcançar esse objetivo, pois penaliza o modelo quando ele estima uma probabilidade baixa para uma classe-alvo. A entropia cruzada é muitas vezes usada com o intuito de calcular o quão bem um conjunto de probabilidades de classe estimadas corresponde às classes-alvo.

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

- Nessa equação temos: A probabilidade-alvo de que a i -ésima instância pertence à classe k . Em geral, é igual a 1 ou 0, dependendo se a instância pertence à classe ou não.



5.2. Regressão softmax

- O vetor gradiente dessa função custo é dada por :

$$\nabla_{\boldsymbol{\theta}}^{(k)} J(\boldsymbol{\Theta}) = \frac{1}{m} \sum_{i=1}^m \left(\hat{p}_k^{(i)} - y_k^{(i)} \right) \mathbf{x}^{(i)}$$

- Agora podemos calcular os gradiente para todas as classes de utilizar o gradiente descendente (ou qualquer outro algoritmo de otimização) a fim de encontrar a matriz de parâmetros que minimizar a função de custo.

